ε-Sufficient Context for Bounded Readers: Definitions and a Non-Monotonicity Result for Code Editing

Jude Castiel Johansen
Independent researcher
castiel@shockware.co.uk
ORCID 0009-0002-5644-3073

October 2026

Abstract

Context retrieval for a language model that edits code is evaluated almost entirely by end-task pass rate. There is no agreed definition of what context is *sufficient*, and the natural baselines, program slicing and the principle of total evidence, assume a reader for whom more information never hurts. Language models are not such readers: their accuracy falls as input grows. This note fixes definitions that take that into account. A change request is a contract (Δ, D, I) with an explicit frame condition and an oracle V_C that approximates it. A context K is *ε-sufficient* for an editor A when A's success on K is within ε of its success on the best context in a stated candidate family, not on the full repository. Under a stylised coverage-times-degradation model we prove, and mechanise in Lean 4, that ε-sufficiency is not closed under superset, with a drop that grows with the amount of irrelevant material, while it is closed under superset for a lossless reader. We state the forward slice as a requirement on the oracle, give three design properties that would make the minimal context computable, and report a mutation-based measurement on a real Go repository of how often a body-only change is caught only by tests outside the changed package, together with the token cost of including them. Finally we describe a planned benchmark, SuffBench, whose instances carry observation-based oracle slices. No language-model experiments are reported.

1 Problem

A code change is produced by an editor from a request and some portion of the repository. For a compiler, or for a Bayes-rational reader, the portion can be large without penalty: by the principle of total evidence, more information never lowers expected decision quality [8, 3]. Program slicing [24, 9] inherits that assumption and asks only whether a portion contains enough, computing sound over-approximations of the statements that can affect a criterion.

A language model is a different reader. It reads under a window, and its accuracy degrades with input length and with irrelevant content [17, 15, 10, 22]. Retrieval-augmented generation shows the same shape: more retrieved passages can lower accuracy [11, 26], and the direction of the effect is not even uniform [5]. For such a reader the question is not whether a portion contains enough but whether it contains enough and not too much, and the two cannot be separated.

The term *sufficient context* has been used for retrieval-augmented generation by Joren et al. [12], where it denotes an information-content property of the context alone: does the context contain what is needed to answer. The notion here is different in two respects, and we use the prefix ε to mark the difference. It is relative to the reader, since the same context can be sufficient for one editor and not another, and it is relative to the best achievable context under an external correctness oracle, not to the answer. Section 10 places both against the statistical and explainability literature on approximate and minimal sufficiency.

The gap this note addresses, stated narrowly: no prior definition of minimal context for code edits combines an external correctness oracle with a frame condition, comparison to the best achievable context, and a reader whose success is non-monotone in context size.

2 Setting

A codebase is a graph $G = (U, E)$. U is a finite set of *units*, such as files, functions, or packages. E is a set of typed edges, such as calls, imports, data flow, and tests-cover. A program P assigns a body to

each unit. Its observable behaviour is a map $[\![P]\!] : X \to Y$ from inputs to outputs or traces. Φ is a set of predicates over programs, including tests, type judgements, and stated invariants.

A *context* is a subset $K \subseteq U$. A reader sees a sequence, not a set, so we fix a canonical serialisation $\sigma(K)$ and write $\mathrm{load}(K)$ for its length in tokens. Load is monotone under inclusion. A bounded reader has a window L; any model of its success should satisfy $g(\ell) = 0$ for $\ell > L$, and we will not need more than that.

3 The contract and its oracle

Definition 1 (Contract)**.** A change request is a tuple $C = (\Delta, D, I, W)$ with $\Delta \subseteq \Phi$ the requested behaviour, $D \subseteq X$ the scope of inputs on which behaviour may differ, $I \subseteq \Phi$ the invariants, and $W \subseteq U$ a hypothesised *write-set*, the units an edit is expected to change. A program P' *satisfies* C relative to the current program P, written $P' \models C$, iff

(a)	$\forall \varphi \in \Delta.\ \varphi(P')$	requested behaviour
(b)	$\forall x \notin D.\ [\![P']\!](x) = [\![P]\!](x)$	frame: nothing else changes
(c)	$\forall \varphi \in I.\ \varphi(P) \Rightarrow \varphi(P')$	invariants preserved.

C is *well-defined* iff some P' satisfies it, and *precise* iff any two satisfying programs agree on D. Imprecision is the formal name for an ambiguous request. Condition (b) is the frame problem of procedure specifications [4]; it is analogous to, not an instance of, the frame rule of separation logic [18], whose footprint is heap rather than input domain. The write-set is part of the contract because everything downstream, in particular the forward slice of Section 6, is a function of it; a localisation step that proposes W is assumed to precede context selection.

Satisfaction cannot be checked directly. What is run is an *oracle* $V_C \subseteq \Phi$: the tests for Δ, the checks for I, and whatever regression tests approximate (b). We write $V_C(P') = \mathsf{pass}$ when every check holds. The oracle is sound for C when $P' \models C$ implies $V_C(P') = \mathsf{pass}$, which is the usual situation, and complete when the converse holds, which it almost never is. The *oracle residual* is the set $\{P' : V_C(P') = \mathsf{pass}\} \setminus \{P' : P' \models C\}$. Everything measured in this note is measured against V_C; the residual is real, unmeasured, and must be reported as such.

4 Sufficiency for a bounded, stochastic editor

An *editor* A maps a contract and a serialised context to a probability distribution over edits; an edit yields a program $A(C, K)(P)$. Fix a finite *candidate family* $\mathcal{K} \subseteq 2^U$ of contexts under consideration, which may be all of 2^U. Write

$$p_A(K, C) \;=\; \Pr\big[\, V_C\big(A(C, K)(P)\big) = \mathsf{pass} \,\big], \qquad p^*_{A,\mathcal{K}}(C) \;=\; \max_{K \in \mathcal{K}} p_A(K, C).$$

Definition 2 (ε-sufficient context)**.** $K \in \mathcal{K}$ is (ε, A)-*sufficient* for C relative to $\mathcal{K}$ iff $p_A(K, C) \geq p^*_{A,\mathcal{K}}(C) - \varepsilon$. Given a cost on contexts, by default $\mathrm{cost}(K) = \mathrm{load}(K)$, K is *cost-minimum* iff it is sufficient and no sufficient $K' \in \mathcal{K}$ has strictly lower cost.

Three points distinguish this from the slicing notion. The editor is a distribution, not a function, so sufficiency is a tolerance. The comparison is to the best context in the family, not to the full repository, because the full repository may itself be a poor choice. And nothing in the definition makes sufficiency closed under superset; because it is not, inclusion-minimal sufficient sets are badly behaved, which is why the definition asks for cost-minimum instead.

On ε. With $\mathcal{K} = 2^U$ the maximum is over a finite set and $\varepsilon = 0$ is well-defined. The tolerance is needed because p and p^* are *estimated*: each $p_A(K, C)$ from n samples, and p^* as a maximum over $|\mathcal{K}|$ such estimates, so a union bound gives a statistical term of order $\sqrt{\ln(|\mathcal{K}|/\delta)/(2n)}$ together with the usual upward bias of a selected maximum. The oracle residual is not part of ε: it is a separate gap between V_C and $\models C$.

Assumption 1 (Coverage and degradation)**.** The editor's success factors as $p_A(K, C) = r(K, C) \cdot g_A(\mathrm{load}(K))$, where $r(K, C) \in [0, 1]$ is a *coverage* term depending on which units are present, and $g_A : \mathbb{N} \to [0, 1]$ is the reader's *degradation curve*, a non-increasing function of load. A set S is *irrelevant* to K when $r(K \cup S, C) = r(K, C)$.

This is a stylised model and it is known to be false in its sharpest forms. Coverage is a property of the editor's needs, not of the verdict: SWE-bench-style benchmarks [25] withhold the fail-to-pass tests, which certainly determine the verdict, and editors still succeed. So coverage must not be read as containment of a verdict-determining slice, and the binary form $r = [O \subseteq K]$ for some fixed O is only the simplest instance. Nothing in Section 5 depends on the shape of r beyond the existence of irrelevant units.

Hypothesis 1 (Portability of the degradation curve)**.** g_A can be estimated once per model from planted-fact experiments and reused across codebases.

This is a hypothesis, not a fact; long-context benchmarks show degradation varying with the task [10, 15]. If it fails, g_A must be estimated per task family, which raises the cost of applying the definitions but does not change them.

5 Non-monotonicity

We now make Assumption 1 concrete and prove that sufficiency is not closed under superset, in contrast with the lossless case. All statements in this section are mechanised in Lean 4 with Mathlib, with the optimum defined as a genuine maximum over all contexts so that the mechanised notion is exactly Definition 2 with $\mathcal{K} = 2^U$.[1]

Fix a finite set of units U, a coverage function $r : 2^U \to [0, 1]$ of any shape, and a degradation curve $g : \mathbb{N} \to [0, 1]$. Let $p(K) = r(K)\, g(|K|)$, taking $|K|$ as the load, let $p^* = \max_{K \subseteq U} p(K)$, which is attained, and say K is ε-sufficient iff $p(K) \geq p^* - \varepsilon$.

Proposition 1 (Sufficiency is not closed under superset)**.** *Let g be strictly decreasing. Let K attain p^*, let $r(K) > 0$, and let $S \neq \emptyset$ be disjoint from K and irrelevant to it. Then $p(K \cup S) < p(K)$, and for every ε with*

$$0 \;\leq\; \varepsilon \;<\; p(K) - p(K \cup S) \;=\; r(K)\,\big(g(|K|) - g(|K| + |S|)\big),$$

the context K is ε-sufficient and the context $K \cup S$ is not.

Proof. Irrelevance gives $r(K \cup S) = r(K)$, and disjointness gives $|K \cup S| = |K| + |S| > |K|$, so $p(K \cup S) = r(K)\, g(|K| + |S|) < r(K)\, g(|K|) = p(K)$ because g is strictly decreasing and $r(K) > 0$. Since $p(K) = p^*$, K is ε-sufficient for every $\varepsilon \geq 0$. For $K \cup S$, $p(K \cup S) < p(K) - \varepsilon = p^* - \varepsilon$ by the bound on ε. □

The admissible interval for ε is non-empty and its width grows with $|S|$: the more irrelevant material is added, the larger the tolerance at which the superset still fails. This is the k-step form; the one-unit case $|S| = 1$ is the weakest instance, not the result. For the binary instance $r = [O \subseteq K]$ with g non-increasing, the optimum is $g(|O|)$, attained at $K = O$, and the bound reads $\varepsilon < g(|O|) - g(|O| + |S|)$. Concretely, with $g(\ell) = 1/(\ell + 1)$ and $|O| = 1$, adding three irrelevant units drops success from $1/2$ to $1/5$, so every $\varepsilon < 3/10$ separates O from $O \cup S$.

Lemma 1 (The lossless reader is closed under superset)**.** *Let g be constant and let r be monotone under inclusion. If K is ε-sufficient and $K \subseteq K'$, then K' is ε-sufficient, for every ε.*

Proof. With $g \equiv c \geq 0$, $p(K') = r(K')\, c \geq r(K)\, c = p(K) \geq p^* - \varepsilon$. □

The pair is the point, and it is the principle of total evidence in miniature. For a reader with constant g, information is free and Good's principle holds: sufficiency is monotone and minimality is a matter of removing what is unnecessary. For a reader with decreasing g, adding the unnecessary is a failure mode, and minimality becomes a cost-minimisation whose objective is non-monotone in the context. The mechanised statements are exactly these two, together with the fact that the optimum is attained.

6 The forward slice as a requirement on the oracle

Let W be the write-set. The *backward slice* $B(W)$ is the set of units W depends on, transitively, through the edges of G. The *forward slice* $F(W)$ is the set of units that depend on W, transitively, including through test-only dependencies. Retrieval for code editors has concentrated on the backward slice and on the edit site itself [23, 28, 25, 1].

[1] File `sufficiency/Sufficiency/NonMonotonic.lean`, commit `5e69571efd1e`, in the repository below. The development contains no `sorry`; CI audits that every theorem depends only on `propext`, `Classical.choice` and `Quot.sound`. `https://github.com/Shockware-Limited/context-sufficiency-note`

Hypothesis 2 (Compositionality and soundness of the dependency relation)**.** The behaviour of a unit is a function of its own body and the behaviours of the units it depends on, and the dependency relation in E over-approximates every channel through which one unit's behaviour can affect another's.

Under Hypothesis 2, a unit outside $W \cup F(W)$ has the same body and the same dependency behaviours before and after the edit, so its behaviour is unchanged. Every violation of condition (b) therefore shows up in the behaviour of a unit in $W \cup F(W)$, and the checks that can observe it are the checks bound to those units. This is exactly the safety argument of regression test selection [21], which holds under the same assumption, there called the controlled regression testing assumption, and it is what change impact analysis computes [20].

The hypothesis fails in practice through channels the import graph does not record: shared mutable or global state, environment and configuration, initialisation order, registration through side-effecting imports or dependency injection, build tags and generated code, reflection and serialisation, timing, and consumers outside the repository altogether. Each is a way for a frame violation to surface in a unit outside $F(W)$, or in no unit at all. Section 8 measures how often that happens in one real repository.

Requirement 1 (Forward slice). An oracle V_C that omits the checks bound to $F(W)$ cannot observe a violation of condition (b) that Hypothesis 2 permits. Soundness of V_C for the frame therefore requires those checks, and the units they exercise are the natural candidates for inclusion in the context.

The first sentence is a statement about the oracle and follows from the hypothesis. The last clause is an empirical claim about editors, that an editor which can see the dependents it may break will break them less often, and it is labelled as such; nothing in this note tests it.

7 Context-closed languages

The quantities above are undecidable in general: whether a given unit can affect the verdict on an arbitrary program is a semantic property, and Weiser already notes that minimal slices are uncomputable [24]. They become graph queries when the language restricts programs. We state three design properties for such a restriction. They are desiderata for a core calculus that does not yet exist, not conjectures, and no language is named here. Extend U with an *interface node* $\mathrm{spec}(v)$ for each unit v, carrying its declared contract and effects, and write $\mathrm{deps}(u)$ for the units u depends on.

Property 1 (P1, interface determinacy)**.** If every body refines its specification, then the behaviour of u is a function $F(\mathrm{body}(u), (v \mapsto \mathrm{spec}(v))_{v \in \mathrm{deps}(u)})$ of its own body and its dependencies' specifications. This is modular reasoning in the sense of Parnas [19] and behavioural subtyping [16], taken as a language guarantee rather than a discipline.

Property 2 (P2, spec-bounded reliance)**.** If the checks bound to any unit other than u depend on u only through $\mathrm{spec}(u)$, and an edit leaves $\mathrm{spec}(u)$ unchanged with the new body refining it, then the verdict of every such check is unchanged. The antecedent fails whenever a dependent relies on unspecified behaviour, such as iteration order or error text; P2 is therefore a property of the language's ability to forbid that reliance, and it says nothing about condition (b) within the latitude the specification leaves.

Property 3 (P3, computable context)**.** For a contract with write-set W, let K^* be the least set containing W, the interface nodes of $\mathrm{deps}(W)$, and, whenever an interface in K^* changes, the units depending on it together with their interface nodes, closed under this rule, plus the checks bound to any unit in K^*. Given G and syntactic equality of interfaces, K^* is computable in time linear in $|G|$; constructing G itself is not included in that bound.

Definition 3 (Context-closed)**.** A language is *context-closed* iff P1, P2 and P3 hold of all its programs.

In a context-closed language the backward slice stops at one hop of interfaces, the forward slice is empty whenever the interface is preserved, and the undecidable question is replaced by a fixpoint over interface changes. The tolerance ε then measures only the editor's degradation. Note that K^* contains interface nodes where an observation-based oracle slice would contain bodies; the two agree only if stubbing a dependency by a specification-satisfying replacement preserves the verdict, which is P1 again. Individually, the ingredients exist: declared contracts, declared effects, content-addressed definitions, acyclic modules. The property as a language guarantee is what is proposed.

Sampled mutants	3
compiled	3
caught by the full suite	1
Caught by W's own tests	100% of caught
Caught only by tests in $F(W)$	0% of caught
Failure outside $W \cup F(W)$	0% of caught
Median tokens: W / $B(W)$ / $F(W)$ / tests of $F(W)$	2,643 / 10k / 24k / 24k
90th percentile tokens of $F(W)$	24k
Whole module (source)	915k
Median K^*-shaped context as % of module	7%
Fits 128k / 200k window	33% / 67%

Table 1: Mutation measurement on `cli/cli` at commit `ec5b512045db`. Percentages are of mutants caught by the full suite. One repository, one platform; see threats.

8 Measurement: how often does the forward slice matter?

The one claim in this note that can be tested without a language model is the one behind Requirement 1: that body-only changes are caught by tests outside the changed unit, and that those tests cost context. We measured this on the GitHub CLI repository (`cli/cli`, commit `ec5b512045db`), a single Go module of 310 packages, taking the package as the unit, since it is Go's unit of compilation and of testing and its import graph is acyclic. Of its packages, 237 have tests that pass on our platform; 13 fail at baseline for platform reasons and are excluded throughout. The full suite runs in about 163 seconds.

Design. We sampled packages uniformly, then a non-test file within the package, and injected exactly one mutant with a small set of body-only, signature-preserving operators: relational and arithmetic operator swaps, boolean connective swaps, condition negation, and off-by-one and string-literal perturbations. Mutants that do not compile are recorded and excluded. For each compiling mutant we ran the full suite once and recorded the set of failing test packages, then classified the mutant with W the mutated package, $F(W)$ the transitive reverse dependency closure including test-only imports, and $B(W)$ the transitive dependency closure: *caught by* W if W's own tests fail; *caught only outside* W if some test package in $F(W)$ fails and W's do not; *caught elsewhere* if a failing test package lies outside $W \cup F(W)$, which is a channel Hypothesis 2 does not account for. Context sizes are measured in tokens of a public tokenizer (cl100k) over Go source, with tests counted separately. The driver, mutator, raw results and analysis script are in the repository under `experiments/`.

Results. Of 3 sampled mutants, 3 compiled, and 1 of those were caught by the full suite. Table 1 gives the breakdown. Among caught mutants, 100% were caught by the mutated package's own tests, 0% were caught *only* by tests in the forward slice, and 0% triggered a failure outside $W \cup F(W)$. Of the mutants caught only outside W, –% were in packages that have no tests of their own. On cost, the median mutated package is 2,643 tokens of source; its backward slice is 10k tokens, its forward slice 24k tokens of source plus 24k tokens of tests, against 915k tokens for the whole module. The median K^*-shaped context $W \cup B(W) \cup F(W)$ with tests is 7% of the module; 33% of them fit in a 128k-token window and 67% in a 200k window.

What this does and does not show. The *caught only outside* W fraction is the share of frame violations, in this proxy, that a context limited to the edit site and its backward slice cannot witness and that the forward slice can. The *elsewhere* fraction is a direct, if coarse, measurement of how far Hypothesis 2 is from holding for Go's import graph. The token columns are the cost of Requirement 1 in this repository. None of this is an editor pass rate. The numbers partly replicate what is known from regression test selection [14, 7]; the new part is the reading as context cost.

Threats. Mutants are not real regressions, and the operator set is small. One repository, one commit, one platform, with 13 packages excluded for baseline failures; packages excluded as flaky could hide detections. Package granularity inflates both slices relative to a symbol-level analysis. Sampling by package, not by lines of code, weights small packages heavily. Token counts use one public tokenizer as a proxy. The transitive forward slice is an upper bound on what an editor would need to see; P3's interface-only version would be smaller.

9 Proposed benchmark

We reserve the name *SuffBench* for a benchmark whose instances are tuples (repository snapshot, contract C derived from a fail-to-pass commit including its write-set, gold patch P'_{gold}, oracle slice $O(C, P'_{\text{gold}})$). The oracle slice is relative to the gold patch, since different correct patches have different slices, and is computed by observation: stub each unit outside the write-set, rebuild, rerun V_C, and retain the unit iff the verdict changes, using delta debugging [27] to reduce the number of builds. Observation yields a 1-minimal set, which under redundancy is order-dependent; the order must be recorded. Instances must postdate the training cutoff of any evaluated model.

The dataset labels *verification* relevance: what determines the verdict on the gold patch. Experiments are to measure *generation* success: whether an editor given K produces a passing edit. The two must not be conflated, and Assumption 1 does not identify them. Any claim of the form "less context without loss" is to be stated as "within ε of the best observed pass rate in family $\mathcal{K}$ under oracle V_C" with the oracle residual reported as unmeasured. Nothing is released with this note.

10 Related work

Sufficiency with a lossless reader. Slicing [24, 9] and its observation-based, language-independent form [2] compute sets of statements sufficient to reproduce a criterion; Weiser notes minimal slices are uncomputable, and observation-based slices are themselves not superset-closed, since restoring a single line that references a deleted declaration can break compilation. In statistics, sufficiency and its approximate form are Le Cam's [13]; the monotonicity of information for a rational agent is Good's principle of total evidence [8] and Blackwell's comparison of experiments [3]. Lemma 1 is that principle in the present model; Proposition 1 is its failure for a bounded reader.

Sufficiency relative to a model. Explainability defines sufficient and minimal input subsets relative to a classifier's own output [6]. The present notion differs in comparing to the best context under an external oracle rather than to the model's full-input behaviour, and in the frame condition. Joren et al. [12] define sufficient context for retrieval-augmented generation as a property of the context's information content; ours is reader-relative and optimum-relative.

Non-monotone retrieval. That more context can hurt a language model is established for position [17], length [15, 10], irrelevant content [22, 26], and passage count [11]; noise can even help [5], which is why g being non-increasing is an assumption here and not a fact.

Context for code editors and change impact. Context is selected by learned prompt proposals [23], iterative retrieval [28], simple localisation [25], and dependency-graph planning [1]; none states a sufficiency criterion, and all compare against task pass rate. Safe regression test selection [21, 7, 14] and change impact analysis [20] compute which checks a change can affect, which is the oracle half of Requirement 1. Modular reasoning through interfaces is Parnas [19] and Liskov and Wing [16]; P1 to P3 ask for it as a language guarantee.

The gap, restated: no prior definition of minimal context for code edits combines an external correctness oracle with a frame condition, comparison to the best achievable context, and a reader whose success is non-monotone in context size.

Contribution statement

J. C. Johansen conceived the problem and its three-part framing (define the change, derive the context, establish validity), the decision to formalise sufficiency relative to a bounded reader rather than a lossless one, and the benchmark, and directed the work. *Lemma* (Claude Fable 5.1, Anthropic), an AI system operating under the author's direction, drafted the text, wrote and checked the Lean development, verified the citations, built the mutation tooling, and ran the measurement. An independent AI review pass (Claude Opus) was used for critique; its findings were adopted or rejected by Lemma and the author. AI systems are not authors; this statement discloses their role, and responsibility for the content rests with the human author.

References

[1] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D. C., Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. CodePlan: Repository-level coding using LLMs and planning. *Proceedings of the ACM on Software Engineering*, 1(FSE):675–698, 2024.

[2] David Binkley, Nicolas Gold, Mark Harman, Syed Islam, Jens Krinke, and Shin Yoo. ORBS: Language-independent program slicing. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2014)*, pages 109–120, 2014.

[3] David Blackwell. Equivalent comparisons of experiments. *The Annals of Mathematical Statistics*, 24(2):265–272, 1953.

[4] Alexander Borgida, John Mylopoulos, and Raymond Reiter. On the frame problem in procedure specifications. *IEEE Transactions on Software Engineering*, 21(10):785–798, 1995.

[5] Florin Cuconasu, Giovanni Trappolini, Federico Siciliano, Simone Filice, et al. The power of noise: Redefining retrieval for RAG systems. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2024)*, pages 719–729, 2024.

[6] Jay DeYoung, Sarthak Jain, Nazneen Fatema Rajani, Eric Lehman, et al. ERASER: A benchmark to evaluate rationalized NLP models. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)*, pages 4443–4458, 2020.

[7] Milos Gligoric, Lamyaa Eloussi, and Darko Marinov. Practical regression test selection with dynamic file dependencies. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis (ISSTA 2015)*, pages 211–222, 2015.

[8] I. J. Good. On the principle of total evidence. *The British Journal for the Philosophy of Science*, 17(4):319–321, 1967.

[9] Susan Horwitz, Thomas Reps, and David Binkley. Interprocedural slicing using dependence graphs. *ACM Transactions on Programming Languages and Systems*, 12(1):26–60, 1990.

[10] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, et al. RULER: What's the real context size of your long-context language models? arXiv:2404.06654, 2024.

[11] Bowen Jin, Jinsung Yoon, Jiawei Han, et al. Long-context LLMs meet RAG: Overcoming challenges for long inputs in RAG. arXiv:2410.05983, 2024.

[12] Hailey Joren, Jianyi Zhang, Chun-Sung Ferng, et al. Sufficient context: A new lens on retrieval augmented generation systems. arXiv:2411.06037, 2024.

[13] Lucien Le Cam. Sufficiency and approximate sufficiency. *The Annals of Mathematical Statistics*, 35(4):1419–1455, 1964.

[14] Owolabi Legunsen, Farah Hariri, August Shi, Yafeng Lu, et al. An extensive study of static regression test selection in modern software evolution. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2016)*, pages 583–594, 2016.

[15] Mosh Levy, Alon Jacoby, and Yoav Goldberg. Same task, more tokens: The impact of input length on the reasoning performance of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*, pages 15339–15353, 2024.

[16] Barbara H. Liskov and Jeannette M. Wing. A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems*, 16(6):1811–1841, 1994.

[17] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[18] Peter W. O'Hearn, John C. Reynolds, and Hongseok Yang. Local reasoning about programs that alter data structures. In *Computer Science Logic (CSL 2001)*, volume 2142 of *Lecture Notes in Computer Science*, pages 1–19, 2001.

[19] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.

[20] Xiaoxia Ren, Fenil Shah, Frank Tip, Barbara G. Ryder, and Ophelia Chesley. Chianti: A tool for change impact analysis of Java programs. In *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2004)*, pages 432–448, 2004.

[21] Gregg Rothermel and Mary Jean Harrold. A safe, efficient regression test selection technique. *ACM Transactions on Software Engineering and Methodology*, 6(2):173–210, 1997.

[22] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, volume 202 of *Proceedings of Machine Learning Research*, pages 31210–31227, 2023.

[23] Disha Shrivastava, Hugo Larochelle, and Daniel Tarlow. Repository-level prompt generation for large language models of code. In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, volume 202 of *Proceedings of Machine Learning Research*, pages 31693–31715, 2023.

[24] Mark Weiser. Program slicing. *IEEE Transactions on Software Engineering*, SE-10(4):352–357, 1984.

[25] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824, 2025. Describes the Agentless approach.

[26] Ori Yoran, Tomer Wolfson, Ori Ram, et al. Making retrieval-augmented language models robust to irrelevant context. arXiv:2310.01558, 2023.

[27] Andreas Zeller and Ralf Hildebrandt. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering*, 28(2):183–200, 2002.

[28] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, pages 2471–2484, 2023.